درس ابزار دقیق، دکتر نیری

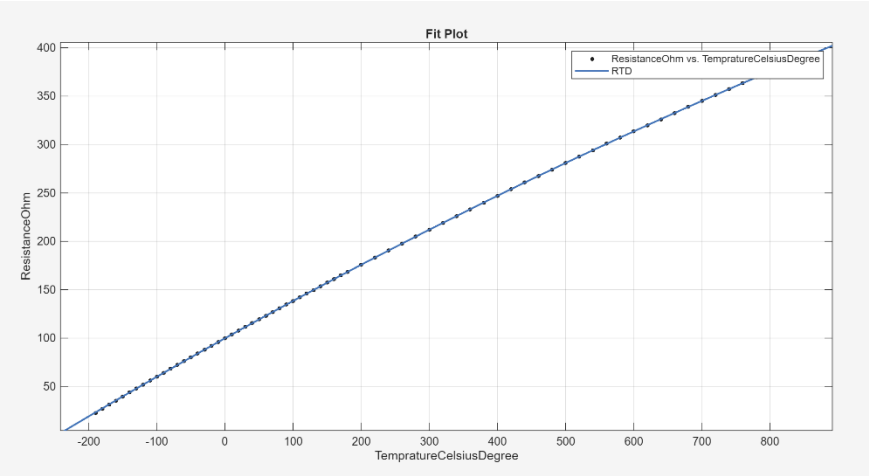
تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

سوال اول

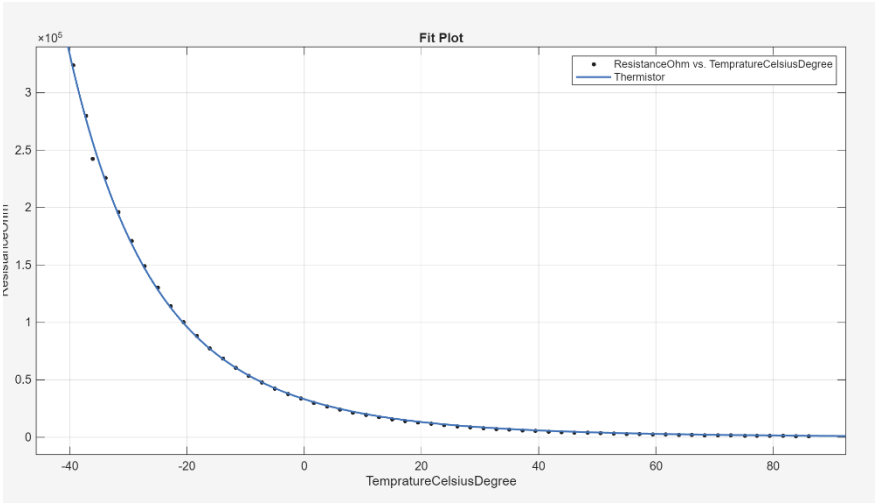
۱- تشخیص حسگر

Dataset 1:



دیتاست اول مربوط به سنسور RTD است. واحد خروجی اهم است. خروجی سنسور RTD نزدیک به خطی است. (ترم‌های درجه بالاتر ضریب بسیار کوچکی دارند.)

Dataset 2:



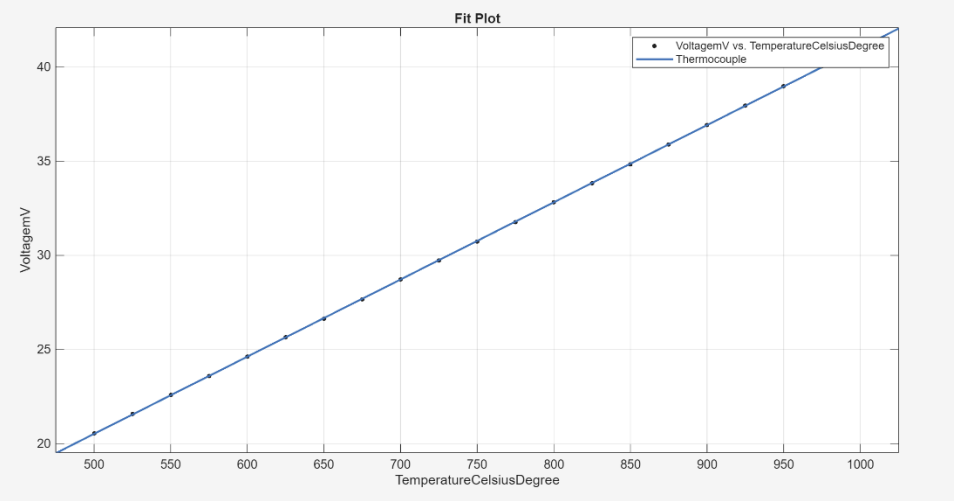
درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

دیتاست دوم مربوط به سنسور ترمیستور است. خروجی سنسور ترمیستور فرم نمایی دارد و از نوع مقاومت است. همچنین سنسور از نوع NTC است. زیرا شیب آن منفی است.

Dataset 3:



دیتاست سوم مربوط به ترموکوپل است. ترموکوپل خروجی ولتاژ دارد و خروجی اش خطی است. همانطور که مشاهده می‌شود این برای نمودار دیتاست سوم برقرار است.

درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

۲- حسگر دمای RTD

۱. پارامترهای استخراج شده متلب:

Polynomial Curve Fit (poly2)

f(x) = p1*x^2 + p2*x + p3

Coefficients and 95% Confidence Bounds

	Value	Lower	Upper
p1	-0.0001	-0.0001	-0.0001
p2	0.3917	0.3914	0.3919
p3	99.8857	99.8471	99.9243

$$R = R_0(1 + \alpha_1 T + \alpha_2 T^2)$$

$$R_0 = 99.8857\Omega, \alpha_1 = 3.921 * 10^{-3}, a_2 = -1.001 * 10^{-6}$$

۲. سنسور داده شده، یک سنسور PT100 است. مقاومت آن در ۰ درجه سانتی گراد ۹۹.۸۸ اهم است که بسیار نزدیک به ۱۰۰ اهم است. همینطور شیب داده‌ها بسیار نزدیک به شیب استاندارد پلاتینوم است. $(\alpha_1 = 3.921 * 10^{-3}, a_2 = -1.001 * 10^{-6})$ پس نوع سنسور قطعا از نوع PT100 است.

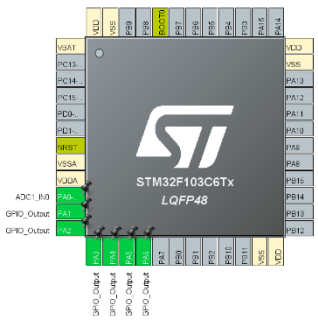
۳. برای تقویت سیگنال RTD از یک instrumentation amplifier استفاده می‌کنیم. برای ساختن instrumentation amplifier میتوان از یک AD620 استفاده کرد.

برای تعیین بهره آن، از یک مقاومت ۹۹ اهم استفاده می‌کنیم. طبق رابطه بهره تقویت کننده ابزار دقیق:

$$Gain = 10 = 1 + \frac{49.4k}{R_G} \rightarrow R_G = 5.49k\Omega$$

همانطور که نتایج نشان می‌دهد تقویت کننده ابزار دقیق به درستی عمل می‌کند و بهره ۱۰ را نشان می‌دهد.

۴. برای پیاده سازی مدار باید از STM32 به عنوان ADC استفاده کنیم. پین ورودی ADC و خروجی‌های متصل به LCD را مشخص می‌کنیم.



داخل کد پین‌های متصل به LCD را تعریف می‌کنیم و آن را تعریف و initialize می‌کنیم.

درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

```
MX_ADC1_Init();
/* USER CODE BEGIN 2 */
Lcd_PortType ports[] = {GPIOA, GPIOA, GPIOA, GPIOA};
  Lcd_PinType pins[] = {GPIO_PIN_3, GPIO_PIN_4, GPIO_PIN_5, GPIO_PIN_6};

  // Create and initialize LCD instance (RS=PA1, EN=PA2)
  Lcd_HandleTypeDef lcd = Lcd_create(ports, pins, GPIOA, GPIO_PIN_1, GPIOA, GPIO_PIN_2, LCD_4_BIT_MODE);

  Lcd_cursor(&lcd, 0, 0);
  Lcd_string(&lcd, "RTD Sensor Init");
  HAL_Delay(1000);
  Lcd_clear(&lcd);
/* USER CODE END 2 */
```

در حلقه اصلی برنامه*(while(1)*، واحد *ADC* میکروکنترلر را راه‌اندازی کرده و منتظر می‌مانیم تا تبدیل سیگنال آنالوگ به دیجیتال کامل شود. مقدار دیجیتال خوانده‌شده یک عدد بین ۰ تا ۴۰۹۵ (به دلیل رزولوشن ۱۲ بیتی *ADC*) خواهد بود.

برای به دست آوردن ولتاژ واقعی، مقدار دیجیتال *ADC* را بر حداکثر مقدار ممکن (۴۰۹۵) تقسیم می‌کنیم. چون در شبیه‌ساز *Proteus* ولتاژ مرجع *(VREF)* میکروکنترلر ۵ ولت تنظیم شده است، نتیجه را در ۵.۰ ضرب می‌کنیم تا ولتاژ دقیق خروجی آپ‌امپ به دست آید.

با توجه به اینکه جریان منبع تغذیه سنسور ۱ میلی‌آمپر است و سیگنال توسط آپ‌امپ *AD620* با بهره *(Gain)* برابر با ۱۰ تقویت شده است، مقدار مقاومت سنسور به سادگی از رابطه $R = V_{ADC} * 100$ به دست می‌آید. سپس با استفاده از ضریب دمایی استاندارد سنسور *PT100 (0.385)* ، دمای محیط بر حسب سانتی‌گراد محاسبه می‌شود.

در نهایت، با استفاده از تابع *sprintf* مقادیر محاسبه‌شده برای ولتاژ، مقاومت و دما را به صورت متن *(String)* قالب‌بندی می‌کنیم. مکان‌نمای *LCD* را تنظیم کرده و ولتاژ را در خط اول، و مقاومت و دما را در خط دوم نمایش می‌دهیم.

درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

```
  while (1)
    {
      /* USER CODE END WHILE */
      /* USER CODE BEGIN 3 */

      HAL_ADC_Start(&hadc1);

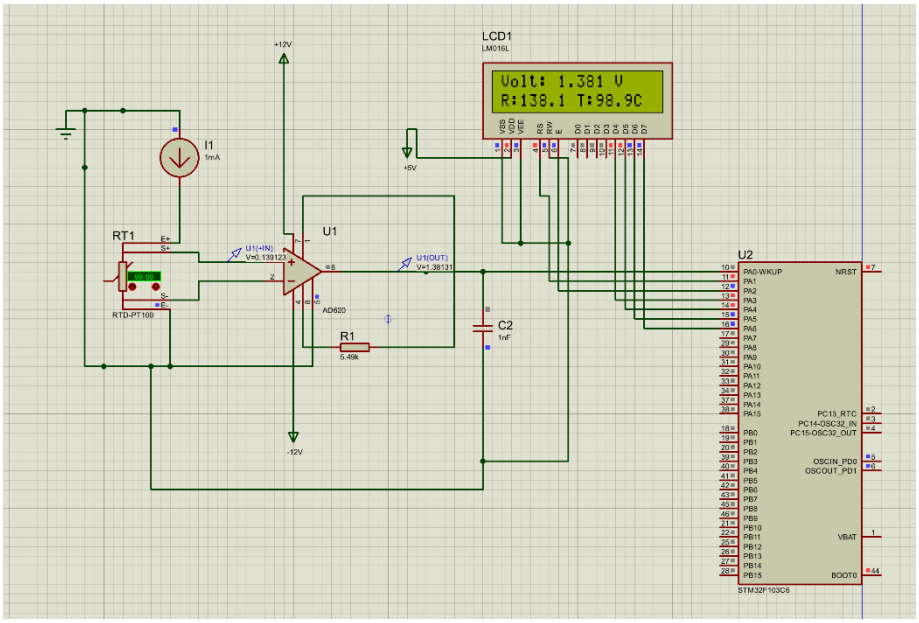
      if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
      {
          uint32_t adc_val = HAL_ADC_GetValue(&hadc1);
          float voltage = ((float)adc_val / 4095.0f) * 5.0f;
          float resistance = voltage * 100.0f;
          float temperature = (resistance - 100.0f) / 0.385f;

          char line1[16];
          char line2[16];

          sprintf(line1, "Volt: %.3f V", voltage);
          sprintf(line2, "R:%.1f T:%.1fC", resistance, temperature);
          Lcd_cursor(&lcd, 0, 0);
          Lcd_string(&lcd, line1);
          Lcd_cursor(&lcd, 1, 0);
          Lcd_string(&lcd, line2);
      }
      HAL_ADC_Stop(&hadc1);

      HAL_Delay(500);
    }
}
  /* USER CODE END 3 */
```

با قرار دادن میکرو در مدار به درستی خروجی نشان داده شد. (اختلاف کوچک با ورودی به دلیل رزولوشن ADC است.)



درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

۳- حسگر دمای ترموکوپل

۱. پارامترهای استخراج شده متلب:

Polynomial Curve Fit (poly1)

f(x) = p1*x + p2

Coefficients and 95% Confidence Bounds

	Value	Lower	Upper
p1	0.0410	0.0409	0.0411
p2	0.0201	-0.0344	0.0745

$$E = \alpha(T - T_0)$$

$$\alpha = 0.0410, \qquad T_0 = 0.4902$$

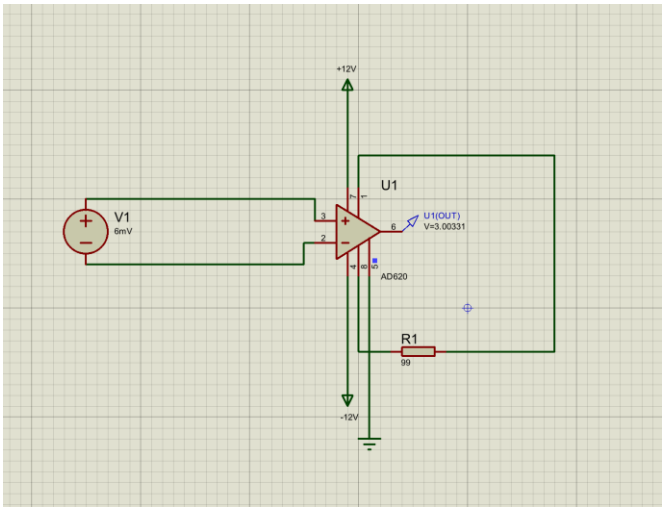
ترموکوپل از نوع K است.زیرا حساسیت آن به ولتاژ $0.04\ \mathrm{mV/°C} \times 1000 = 40\mu\mathrm{V/°C}$ است و این مشخصه نشانگر ترموکوپل نوع K است.

۲. در این بخش می‌خواهیم ترموکوپل را با یک منبع ولتاژ DC نشان دهیم. دمای ۱۵۰ درجه را در نظر می‌گیریم:

$$V_{in} = 40\mu\mathrm{V/°C} \times 150\mathrm{°C} = 6000\mu\mathrm{V} = 6\ \mathrm{mV}$$

۳. مانند بخش قبل از AD620 استفاده می‌کنیم. فرض کنیم gain=500:

$$Gain = 500 = 1 + \frac{49.4}{R_G} \rightarrow R_G = 99\Omega$$



درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

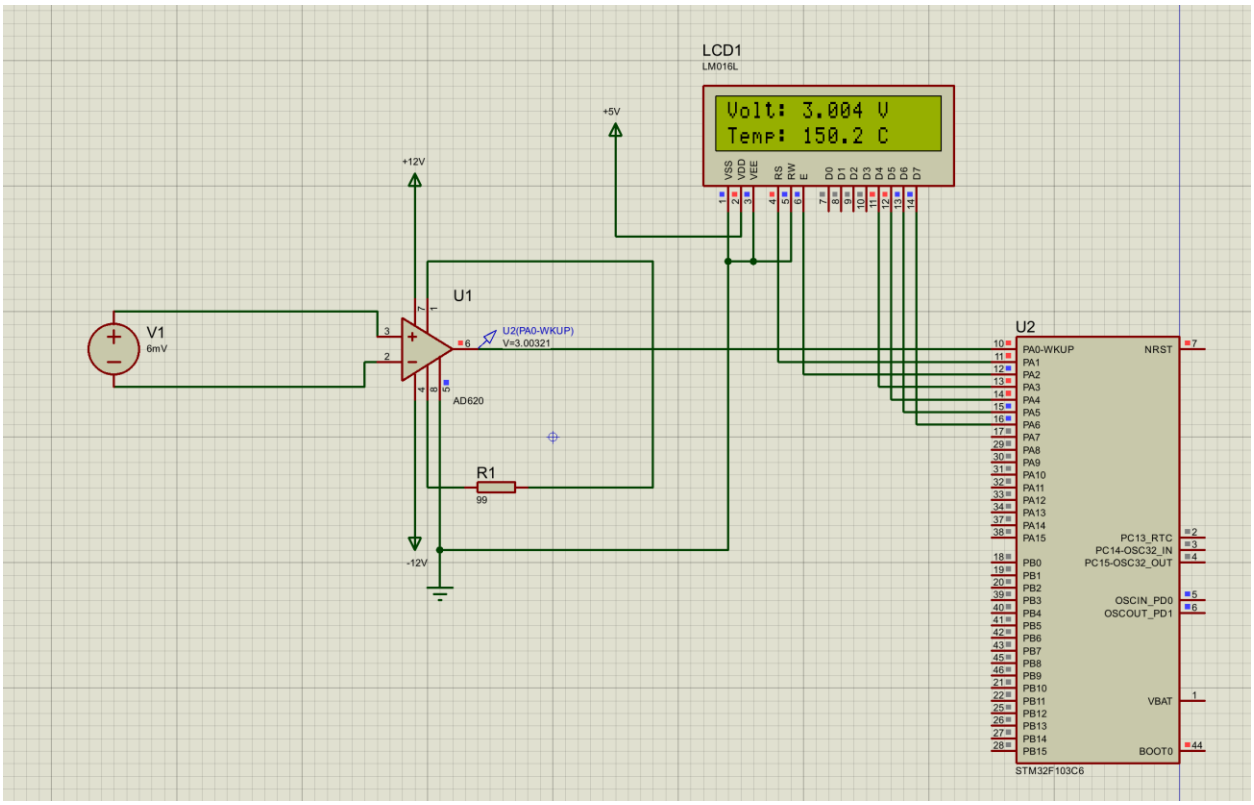
۴. با توجه به مشخصات ترموکوپل نوع K ، حساسیت (ضریب سیبک) آن برابر با $40\mu V/^\circ C$ است. در این مدار از آپامپ ابزار دقیق AD620 با بهره (Gain) برابر با ۵۰۰ استفاده شده است. بنابراین به ازای هر یک درجه سانتی‌گراد، ولتاژ خروجی به این شکل محاسبه می‌شود :

$$40\mu V \times 500 = 20{,}000\mu V = 0.02V$$

پس ولتاژی که به ADC می‌رسد V_{ADC} برابر با $0.02 * T$ خواهد بود. برای به دست آوردن دما در کد، کافی است ولتاژ خوانده‌شده را بر ۰.۰۲ تقسیم کنیم (که معادل ضرب کردن در عدد ۵۰ است).

دقیقاً مانند مدار قبلی، در حلقه اصلی برنامه (while(1)) واحد ADC میکروکنترلر STM32 را استارت زده و منتظر می‌مانیم تا تبدیل آنالوگ به دیجیتال روی پین متصل به خروجی آپامپ کامل شود.

بر اساس فرمول استخراج‌شده در بخش اول، ولتاژ به دست آمده را در عدد ۵۰ ضرب می‌کنیم تا دمای اندازه‌گیری شده توسط ترموکوپل بر حسب سانتی‌گراد به دست آید. همانطور که مشاهده می‌شود خروجی صحت مدار را نشان می‌دهد.



درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

۴- حسگر دمای ترمیستور

۱. پارامترهای استخراج شده متلب:

Custom Curve Fit

f(x) = R0*exp(beta*(1/x-1/298.15))

Coefficients and 95% Confidence Bounds

	Value	Lower	Upper
R0	1.0769e+04	1.0353e+04	1.1185e+04
beta	3.6707e+03	3.6234e+03	3.7180e+03

$$R_0 = 1.0769 * 10^4, \beta = 3.6707 * 10^3$$

با توجه به برازش منحنی *(Curve Fitting)* در نرم‌افزار متلب، معادله مقاومت-دمای ترمیستور با استفاده از مدل بتا β تعریف شده است. در این مدل، دمای مرجع T_0 برابر با ۲۵ درجه سانتی‌گراد (۲۹۸.۱۵K) فرض شده است.

با توجه به اینکه مقاومت پایه در دمای اتاق حدود ۱۰ $k\Omega$ است و با افزایش دما، مقاومت کاهش می‌یابد (شیب منفی)، این سنسور قطعاً یک ترمیستور از نوع *NTC* است. مقادیر به‌دست‌آمده، مقادیر استانداردی برای ترمیستورهای تجاری *NTC 10k* محسوب می‌شوند.

۲. برای راه‌اندازی ترمیستور و خواندن مقادیر آن توسط میکروکنترلر، برخلاف سنسورهای قبلی نیازی به تقویت‌کننده ابزار دقیق (*Instrumentation Amplifier*) نیست. تغییرات مقاومت ترمیستور نسبتاً بزرگ است، بنابراین از یک مدار مقسم ولتاژ (*Voltage Divider*) ساده استفاده می‌کنیم.

در این مدار، ترمیستور با یک مقاومت ثابت (در اینجا ۱۰ $k\Omega$ برای دستیابی به بیشترین حساسیت خطی در حوالی دمای اتاق) سری می‌شود و ولتاژ گره میانی به پین ورودی *ADC* میکروکنترلر متصل می‌گردد.

درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

```
float R0 = 10769.0f;
float beta = 3670.0f;
float T0 = 298.15f;       // 25 degrees Celsius in Kelvin
float R_fixed = 10000.0f; // 10k ohm resistor
float V_ref = 5.0f;       // Proteus ADC reference voltage

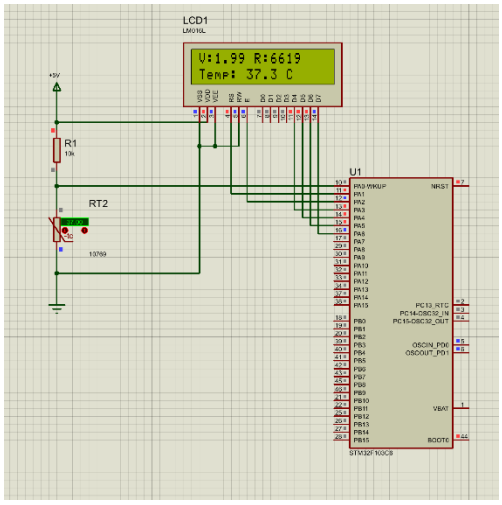
// 1. Convert ADC value to Voltage
float voltage = ((float)adc_val / 4095.0f) * V_ref;

// 2. Calculate Thermistor Resistance (Voltage Divider Formula)
// Prevent division by zero if voltage spikes to exactly 5.0V
float resistance = 0.0f;
if (voltage < V_ref) {
    resistance = (R_fixed * voltage) / (V_ref - voltage);
}

// 3. Calculate Temperature using the Beta Equation
// Note: log() in C computes the natural logarithm (ln)
float temp_k = 1.0f / ((1.0f / T0) + (1.0f / beta) * log(resistance / R0));

// Convert Kelvin to Celsius
float temperature_c = temp_k - 273.15f;
```

در حلقه اصلی برنامه میکروکنترلر، ابتدا مقدار دیجیتال *ADC* خوانده شده و با توجه به ولتاژ مرجع، به ولتاژ واقعی آنالوگ تبدیل می‌شود .سپس با استفاده از رابطه بازگشتی مقسم ولتاژ، مقدار مقاومت لحظه‌ای ترمیستور محاسبه می‌گردد. در مرحله آخر، با استفاده از معکوس معادله مشخصه (مدل بتا) و داشتن پارامترهای استخراج‌شده از متلب، دمای محیط بر حسب کلوین محاسبه شده و پس از تبدیل به درجه سانتی‌گراد، روی نمایشگر *LCD* چاپ می‌شود.



درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

سوال دوم

الف)

تاب آوری سیگنال‌های دیجیتال نسبت به نویز بسیار بیشتر از سیگنال‌های آنالوگ است. پس استفاده از سنسور دیجیتال برای اندازه‌گیری در شرایط نویزی مناسب‌تر است. همچنین در سنسورهای دیجیتال تبدیل سیگنال آنالوگ به دیجیتال داخل سنسور اتفاق می‌افتد و سیگنالی که به میکروکنترلر ارسال می‌شود سیگنال دیجیتال است. اثر نویز روی سیگنال دیجیتال بسیار کمتر از سیگنال آنالوگ است. پس سیگنال دیجیتال از منظر دقت خوانده شده در میکروکنترلر نیز ارجحیت دارد. بهترین پروتکل ارتباطی با میکروکنترلر برای شرایط نویزی حاضر و فاصله زیاد تا میکروکنترلر، پروتکل CAN است. پروتکل CAN یک پروتکل سریالی تفاضلی است که برای ارتباط‌های نویزی صنعتی طراحی شده. سرعت آن به ماکسیمم 1Mbps می‌رسد که در فواصل طولانی (۱ کیلومتر) سرعت آن کمتر است.

ب)

برای خواندن دما از سنسور دیجیتال LM75، باید از رابط I2C میکروکنترلر STM32 (ماژول Bluepill) استفاده کنیم.
ابتدا باید در نرم‌افزار STM32CubeIDE، واحد I2C1 را فعال کنیم.
شبیه‌سازی پروتئوس: سنسور دمای LM75 از طریق باس ارتباطی I2C به میکروکنترلر متصل می‌شود. پین‌های SCL و SDA سنسور به پین‌های متناظر در میکروکنترلر STM32 (پین‌های PB6 و PB7) متصل شده و از مقاومت‌های Pull-up برای عملکرد صحیح این خطوط استفاده شده است.
توضیحات کد: در محیط STM32CubeIDE، واحد I2C1 فعال شده است. در حلقه اصلی برنامه، با استفاده از تابع HAL_I2C_Mem_Read دیتای مربوط به رجیستر دما خوانده می‌شود. از آنجا که LM75 یک سنسور ۹ بیتی است، داده‌ها در ۹ بیت پرارزشِ یک متغیر ۱۶ بیتی ذخیره می‌شوند؛ بنابراین ۹ بیت بالایی استخراج می‌گردد. با توجه به دقت

```
  while (1)
    {
        if (HAL_I2C_Mem_Read(&hi2c1, LM75_ADDR_8BIT, TEMP_REGISTER, I2C_MEMADD_SIZE_8BIT, temp_data, 2, HAL_MAX_DELAY) == HAL_OK)
        {
            raw_temp = (int16_t)((temp_data[0] << 8) | temp_data[1]) >> 7;

            int whole_part = raw_temp / 2;
            int decimal_part = (raw_temp % 2) * 5;

            if(decimal_part < 0) decimal_part *= -1;
            sprintf(display_buf, "Temp: %d.%d C   ", whole_part, decimal_part);
            Lcd_cursor(&lcd, 0, 0);
            Lcd_string(&lcd, display_buf);
        }
        else
        {
            Lcd_cursor(&lcd, 0, 0);
            Lcd_string(&lcd, "I2C Read Error! ");
        }

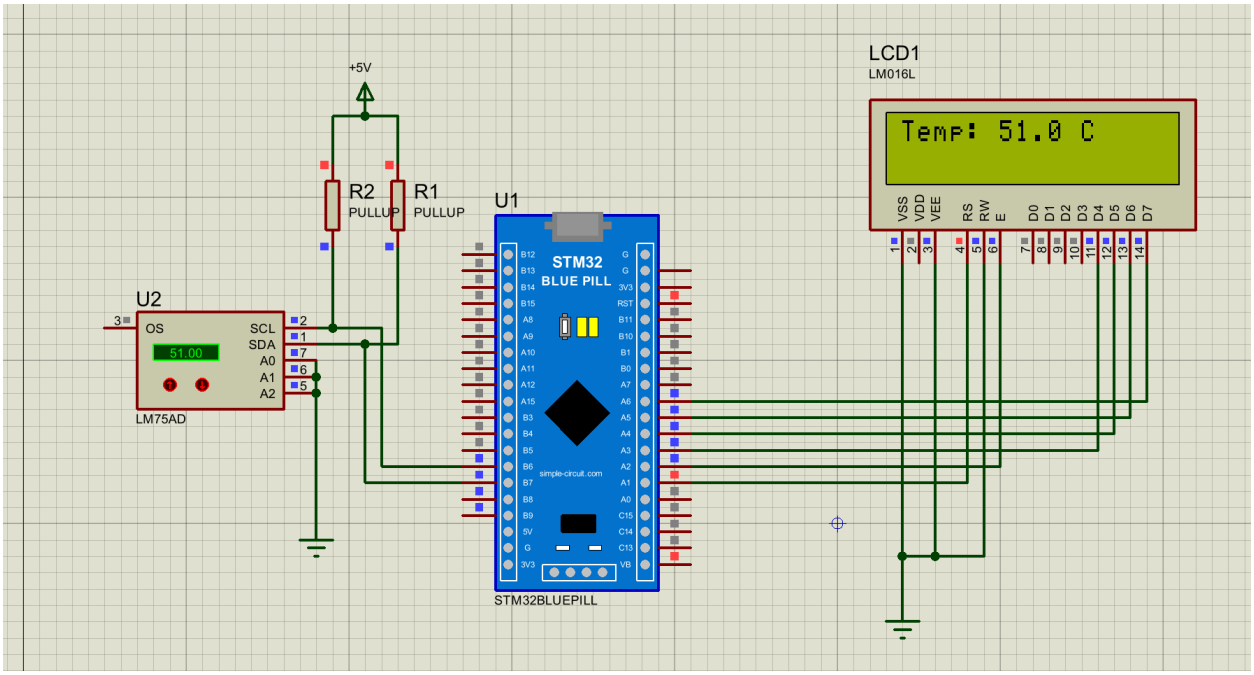
        HAL_Delay(200);
    }
/* USER CODE END 3 */
```

درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

سنسور، مقدار به‌دست‌آمده در عدد ۰.۵ ضرب می‌شود تا دمای نهایی بر حسب سانتی‌گراد محاسبه شده و روی نمایشگر LCD چاپ شود.



درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

سوال سوم

الف)

برای تولید مدار instrumentation amplifier از یک AD620 استفاده می‌کنیم. ولتاژ رفرنس (پین ۵) را زمین قرار می‌دهیم تا در صورت گرفتن ولتاژ تفاضلی ۰، خروجی ۰ بدهد. برای تعیین بهره آن، از یک مقاومت ۹۹ اهم استفاده می‌کنیم. طبق رابطه بهره تقویت کننده ابزار دقیق:

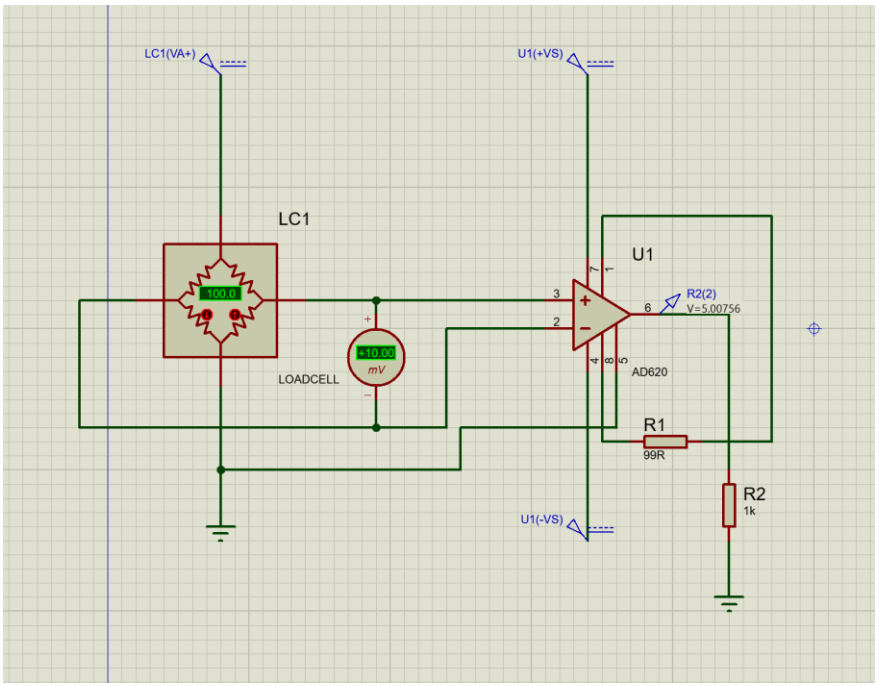
$$Gain = 500 = 1 + \frac{49.4}{R_G} \rightarrow R_G = 99\Omega$$

همانطور که نتایج نشان می‌دهد تقویت کننده ابزار دقیق به درستی عمل می‌کند و بهره ۵۰۰ را نشان می‌دهد.

رابطه جرم و ولتاژ خروجی:

$$Voltage_{output} = mass * 2.5$$

$mass = 2kg \rightarrow V = 5V$

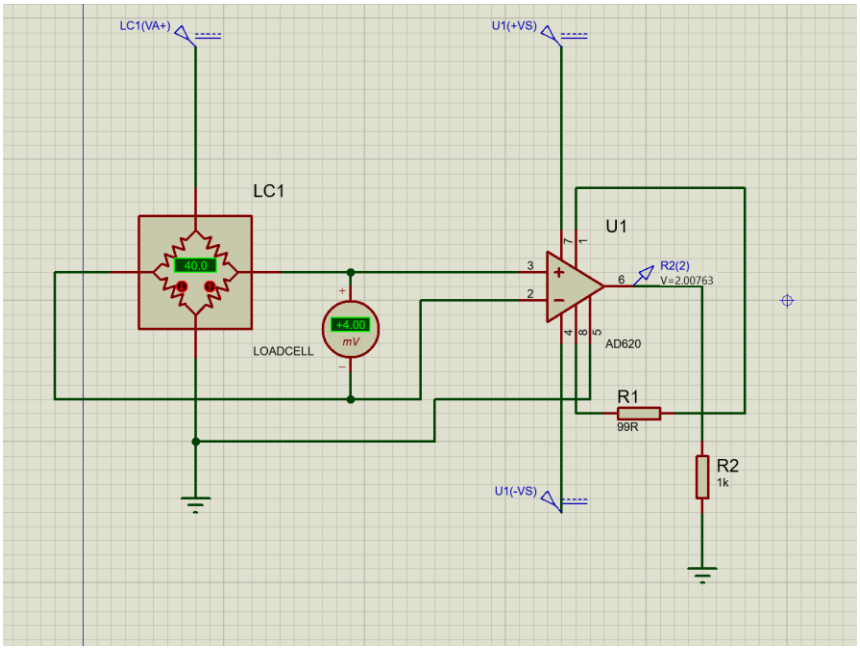


$mass = 0.4 * 2 = 0.8kg \rightarrow V = 2V$

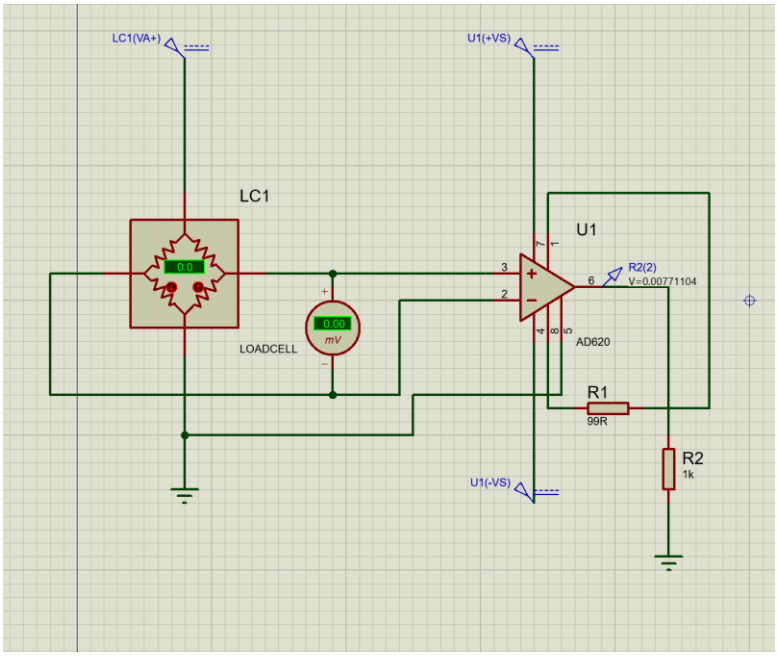
درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸



$mass = 0kg \rightarrow V = 0V$



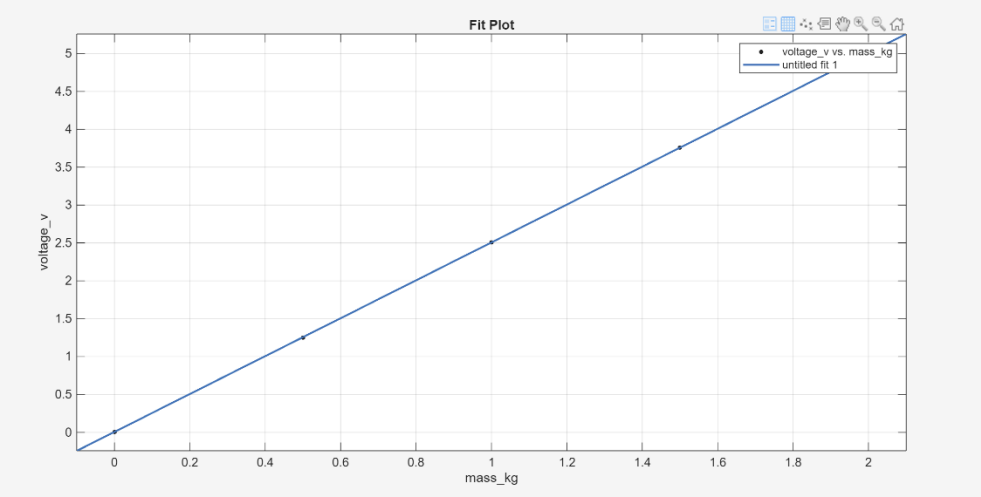
درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

ب)

داده‌های اندازه گیری شده در شبیه سازی ایده آل قسمت الف:



همانطور که مشاهده می‌شود داده‌ها تقریبا به طور دقیق روی منحنی خطی پیش بینی شده قرار گرفته اند:

Polynomial Curve Fit (poly1)

f(x) = p1*x + p2

Coefficients and 95% Confidence Bounds

	Value	Lower	Upper
p1	2.5007	2.4969	2.5044
p2	0.0062	0.0016	0.0107

ج)

برای شبیه‌سازی نوسان از یک فیلتر مرتبه دوم RLC استفاده می‌کنیم.

به جای سنسور لودسل واقعی که خروجی پله‌ای نمی‌دهد، ما یک منبع ولتاژ پالسی (VPULSE) در مدار قرار دادیم تا ضربه ناگهانی افتادن وزنه روی ترازو را مدل کنیم. این پالس با دامنه ۵mV و زمان‌های صعود و نزول ۵ms وارد یک آپامپ ایده‌آل (OPAMP) می‌شود که به صورت تقویت‌کننده غیرمعکوس بسته شده است. مقاومت‌های فیدبک و زمین بهره‌ای معادل ۵۰۱ برای ما فراهم می‌کنند.

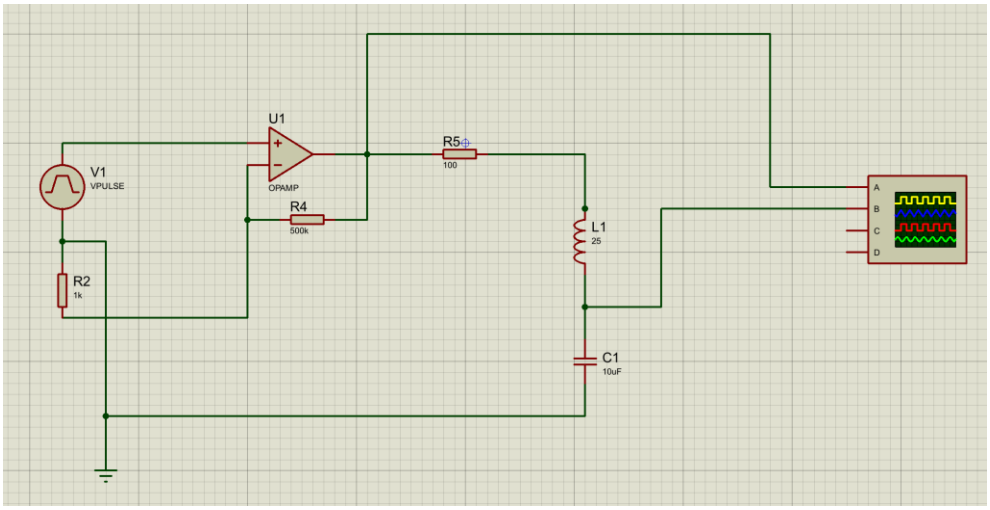
سپس خروجی این آپامپ به یک مدار RLC متشکل از مقاومت ۱۰۰ اهم سلف ۲۵ هانری و خازن ۱۰ میکروفاراد وارد می‌شود. این ترکیب مقادیر، باعث ایجاد یک رفتار نوسانی میرا (Underdamped) در ولتاژ دو سر خازن می‌گردد که دقیقاً شبیه‌ساز رفتار

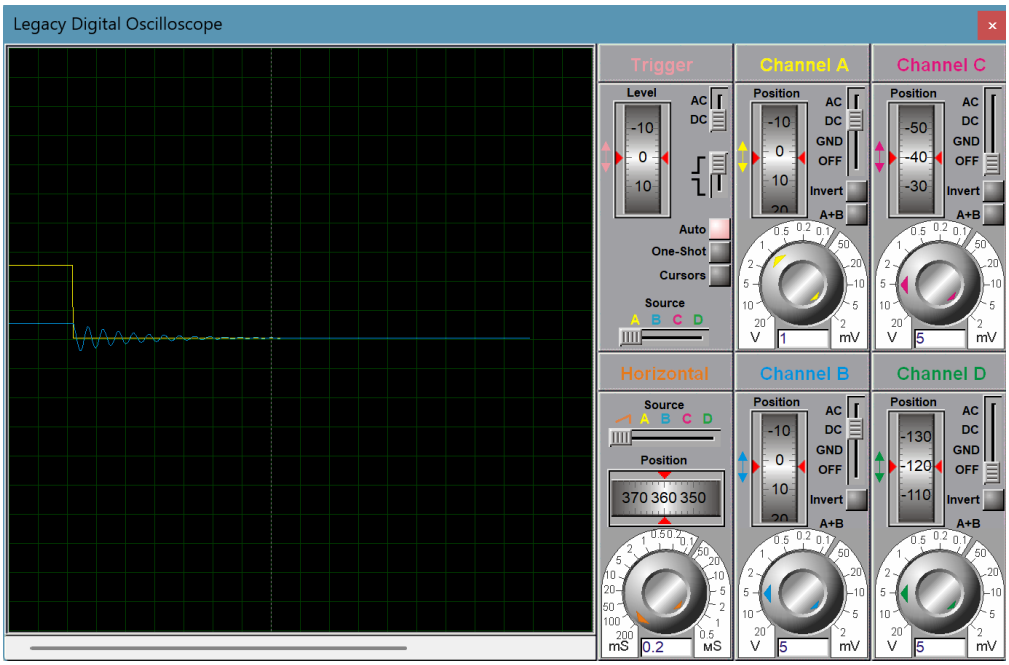
درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

مکانیکی لودسل هنگام قرار گرفتن بار ناگهانی روی آن است. ولتاژ نهایی پس از میرا شدن نوسانات، روی مقدار مورد نظر ثابت می‌ماند.





درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

د)

در بخش (د) هدف ما طراحی یک فیلتر پایین‌گذر (Low-Pass Filter) برای حذف نوسانات فرکانس بالای ناشی از ارتعاشات مکانیکی لودسل است. اگر این نوسانات حذف نشوند، مقادیر خوانده شده توسط ADC میکروکنترلر دائماً تغییر کرده و عدد پایداری روی نمایشگر نشان داده نمی‌شود.

برای این منظور، از یک فیلتر غیرفعال مرتبه اول RC (متشکل از یک مقاومت و یک خازن) در خروجی مدار ارتعاشی استفاده کردیم. طبق صورت سوال، فرکانس قطع (Cut-off frequency) فیلتر باید حدود ۵ هرتز تنظیم شود. با استفاده از رابطه فرکانس قطع:

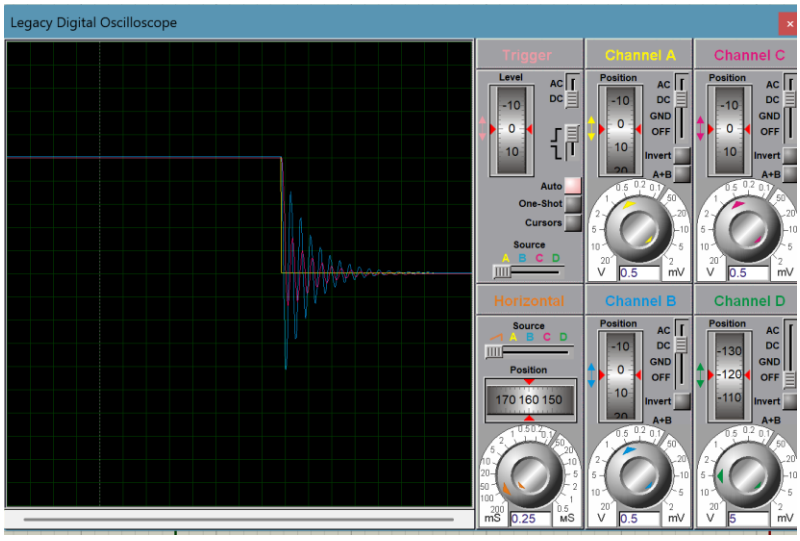
$$f_c = \frac{1}{2\pi RC}$$

با فرض انتخاب یک خازن استاندارد مانند، $1\mu F$, مقدار مقاومت مورد نیاز حدود ۳۱.۸ کیلو اهم محاسبه می‌شود. ما این ترکیب مقاومت و خازن را به خروجی مدار RLC (گره بالای خازن لودسل) متصل کردیم.

همان‌طور که در تصویر اسیلوسکوپ مشاهده می‌شود:

سیگنال آبی (کانال B): نشان‌دهنده ولتاژ خام و نوسانی (ارتعاشات لودسل) است که به سرعت بالا و پایین می‌رود.

سیگنال صورتی (کانال C): خروجی فیلتر پایین‌گذر (LPF) ما را نشان می‌دهد.

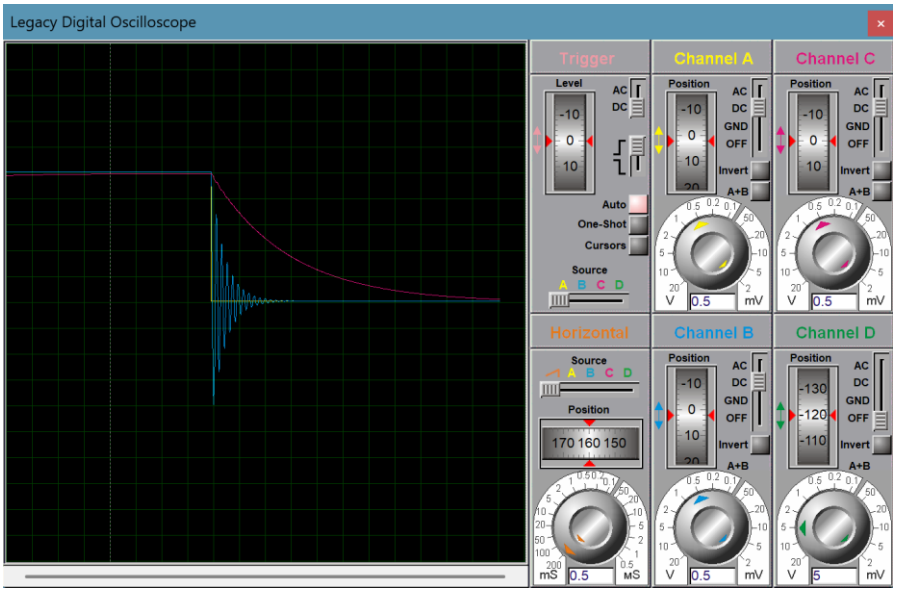


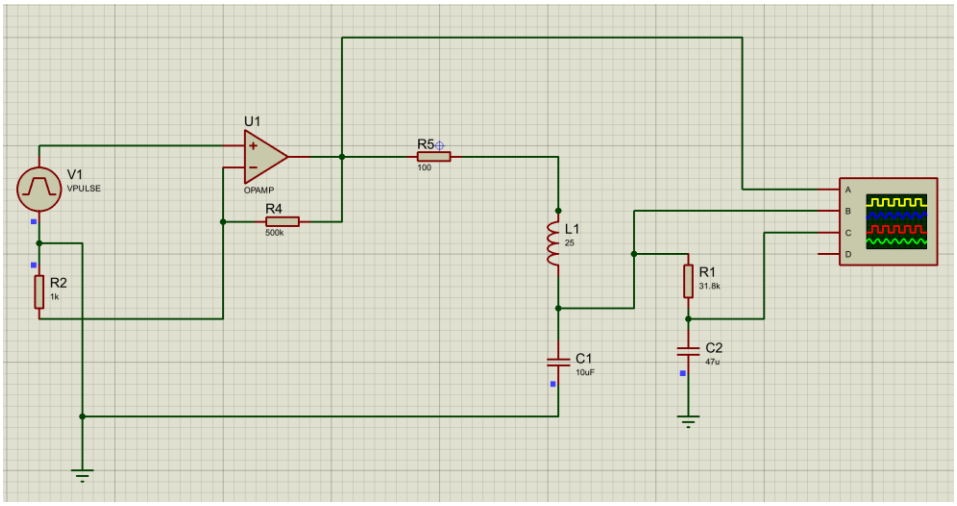
درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

مشاهده می‌کنیم که فیلتر میزان نویزها را کم کرده است اما به طور کامل آن را از بین نبرده، به همین دلیل فرکانس قطع را پایین برده و خازن را به ۴۷ میکروفاراد می‌رسانیم. نتیجه سیگنال بدون ریپل و صاف است.





درس ابزار دقیق، دکتر نیری

تمرین چهارم

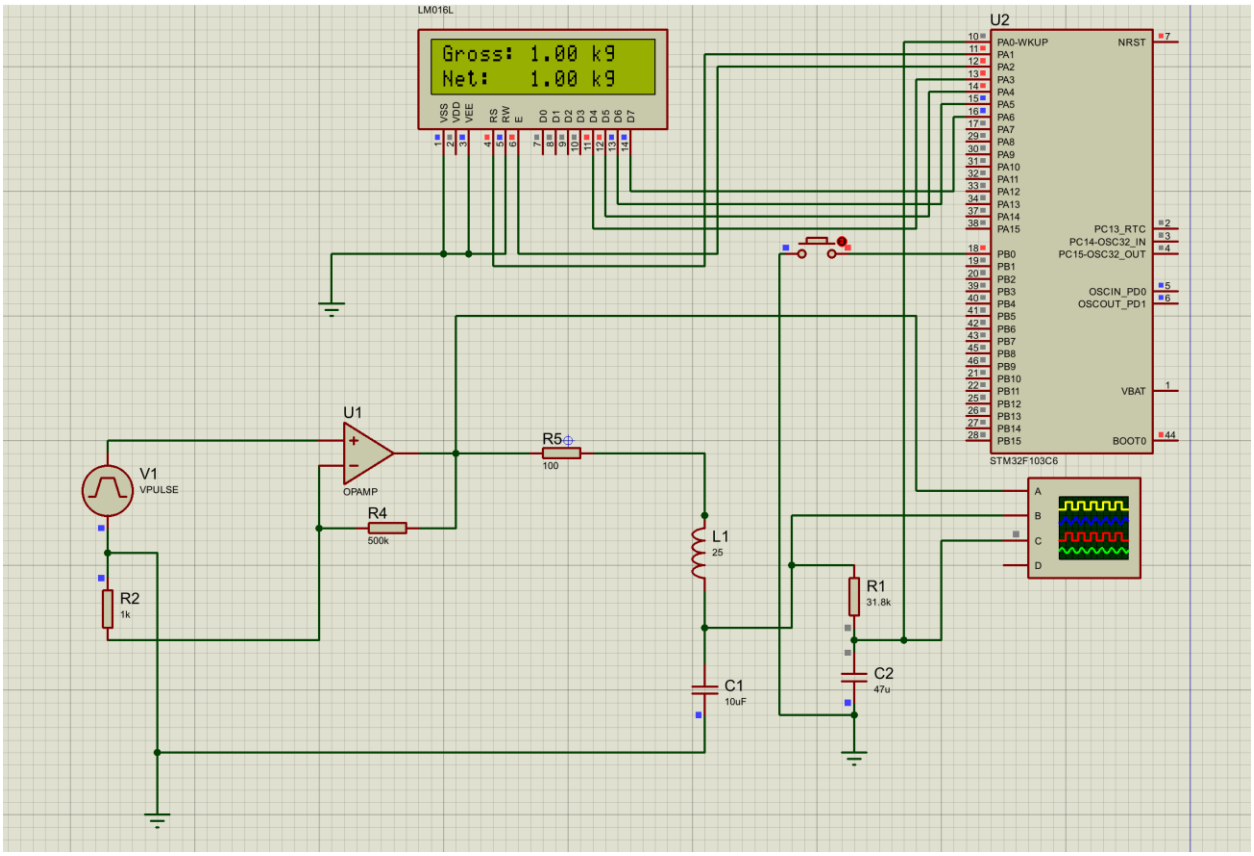
غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

۵)

در این بخش، برنامه میکروکنترلر STM32 برای خواندن ولتاژ فیلتر شده، محاسبه جرم و نمایش آن روی LCD نوشته شده است. پین PA0 به عنوان ورودی آنالوگ (ADC1_IN0) برای خواندن خروجی فیلتر پایین‌گذر، و پین PB0 به عنوان ورودی دیجیتال با مقاومت Pull-up داخلی برای کلید Tare تنظیم گردید. پین‌های PA1 تا PA6 نیز برای ارتباط ۴ بیتی با LCD تخصیص داده شدند. در حلقه اصلی برنامه، ابتدا واحد ADC راه‌اندازی شده و ولتاژ آنالوگ خوانده می‌شود. با استفاده از رابطه خطی محاسبه شده در بخش (ب) $V = 2.5 * mass$، ولتاژ به جرم (Gross Mass) تبدیل می‌گردد.

برای پیاده‌سازی قابلیت Tare، وضعیت پین PB0 بررسی می‌شود. در صورت فشرده شدن کلید (صفر شدن پین)، مقدار فعلی جرم به عنوان tare_offset ذخیره می‌شود. در نهایت، جرم خالص (Net Mass) از تفاضل جرم کل و مقدار آفست محاسبه شده و هر دو مقدار روی نمایشگر چاپ می‌شوند. این ساختار به ما اجازه می‌دهد تا با فشردن یک دکمه مجازی در پروتئوس، ترازو را صفر کنیم.

در خروجی همانطور که انتظار می‌رفت با مقداری تاخیر و به صورت تدریجی مقدار به مقدار وزن می‌رسد و دارای نوسان نیست.



درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

و)

شکل مورد نظر و توضیحات به طور کامل در قسمت د داده شده.

ی)

زمان تقریبی رسیدن خروجی یک فیلتر پایین‌گذر RC مرتبه اول به ۹۵٪ مقدار نهایی خود، معادل **سه برابر ثابت زمانی فیلتر** $\boldsymbol{3\tau}$ است .ثابت زمانی از رابطه $\tau = RC$ به دست می‌آید .

با توجه به مقادیری که برای رسیدن به یک منحنی کاملاً صاف در پروتئوس استفاده کردیم:

$$\tau = RC = 31800 * 47 * 10^{-6} = 1.49s$$

$$3\tau = 4.47s$$

این یعنی ترازو برای نشان دادن وزن دقیق و نهایی، حدود ۴.۵ ثانیه زمان نیاز دارد که برای یک ترازوی دیجیتال صنعتی زمان نسبتاً کندی محسوب می‌شود.

آیا می‌توان این زمان را بهبود داد؟ بله، قطعاً می‌توان این زمان (Settling Time) را کاهش داد بدون اینکه کیفیت فیلتر کردن نوسانات افت کند .

چگونه؟ چندین روش مهندسی برای این کار وجود دارد :

۱. **استفاده از فیلترهای اکتیو مرتبه بالاتر :(Higher-Order Active Filters)** فیلتر RC که استفاده کردیم از نوع مرتبه اول است و شیب افت ملایمی دارد. اگر از یک فیلتر مرتبه دوم یا سوم (مثلاً فیلتر باتروورث با آپ‌امپ) استفاده کنیم، شیب قطع بسیار تندتر می‌شود. این یعنی می‌توانیم فرکانس قطع را بالاتر ببریم (که باعث سرعت بیشتر سیستم می‌شود) در حالی که همچنان نوسانات ۲۰ هرتزی لودسل به شدت فیلتر و حذف می‌شوند.

۲. **استفاده از فیلتر متغیر با زمان :(Time-varying Filter)** همان‌طور که در نمودار صورت سوال (بخش ج) با خط ممتد مشکی نشان داده شده بود، می‌توان فیلتری طراحی کرد که به محض تغییرات بزرگ (مثل افتادن وزنه) ثابت زمانی کوچکی داشته باشد تا سریعاً به مقدار نهایی نزدیک شود، و سپس ثابت زمانی آن بزرگ شود تا نوسانات ریز را به شدت فیلتر کند.

۳. **فیلتر دیجیتال درون میکروکنترلر :(Digital Filtering)** به جای اینکه تمام بار فیلتر کردن را روی دوش مدار آنالوگ بیندازیم، می‌توانیم یک فیلتر سخت‌افزاری سریع‌تر (با خازن کوچکتر) ببندیم و سپس در کد STM32 از

درس ابزار دقیق، دکتر نیری

تمرین چهارم

غزل شریعت محرری-۸۱۰۱۰۲۴۶۸

الگوریتم‌هایی مثل "میانگین‌گیر متحرک (Moving Average) "یا فیلترهای نرم‌افزاری FIR/IIR استفاده کنیم تا هم سرعت پاسخ‌دهی بالا برود و هم عدد روی LCD کاملاً پایدار بماند.